

Make a sql database,of a game

1. Create a database called GameDB.

Answer:

```
CREATE DATABASE GameDB
```



2. Create an table called item_types with the following item

types: • melee

- ranged
- defense
- potion
- magic

Answer:

```
CREATE TABLE item_types (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50) UNIQUE NOT NULL  
);
```

```
INSERT INTO item_types (name) VALUES  
( 'melee' ),  
( 'ranged' ),  
( 'defense' ),  
( 'potion' ),  
( 'magic' );
```

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0310 seconds.)

CREATE TABLE item_types ( name VARCHAR(50) PRIMARY KEY );

[ Edit inline ] [ Edit ]

✓ 5 rows inserted. (Query took 0.0120 seconds.)

INSERT INTO item_types (name) VALUES ('melee'), ('ranged'), ('defense'), ('potion'), ('magic');

[ Edit inline ] [ Edit ]
```

3. Create a table `items` using `name` as the primary key. Each item has:
- A unique name
 - Durability (default 100)
 - Type (from `item_type_enum`)

Answer:

```
CREATE TABLE items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    durability INT DEFAULT 100,
    item_type VARCHAR(50) NOT NULL,
    FOREIGN KEY (item_type) REFERENCES item_types(name)
);
```

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0135 seconds.)

CREATE TABLE items ( name VARCHAR(50) PRIMARY KEY, durability INT DEFAULT 100, item_type VARCHAR(50) NOT NULL,
FOREIGN KEY (item_type) REFERENCES item_types(name) );

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

4. Create a table `players` with the following:

- `id` (Primary Key)
- `name`
- `health` (default 100)
- `equipped_item_name` (Foreign key referencing `items(name)`)

Answer:

```
CREATE TABLE players (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    health INT DEFAULT 100,
    equipped_item_id INT,
```

```
FOREIGN KEY (equipped_item_id) REFERENCES items(id)
);
```

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0137 seconds.)

CREATE TABLE players ( id SERIAL PRIMARY KEY, name VARCHAR(50) NOT NULL, health INT DEFAULT 100,
equipped_item_name VARCHAR(50), FOREIGN KEY (equipped_item_name) REFERENCES items(name) );

[ Edit inline ] [ Edit ]
```

5. Create a table inventory with:

- id (Primary Key)
- player_id (Foreign key)
- item_name (Foreign key referencing items(name))
- amount (default 1)

Answer:

```
CREATE TABLE inventory (
    id INT AUTO_INCREMENT PRIMARY KEY,
    player_id INT UNSIGNED NOT NULL,
    item_id INT NOT NULL,
    amount INT DEFAULT 1 CHECK (amount >= 0),
    FOREIGN KEY (player_id) REFERENCES players(id) ON DELETE
CASCADE,
    FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE CASCADE
);
```

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0133 seconds.)

CREATE TABLE inventory ( id SERIAL PRIMARY KEY, player_id BIGINT UNSIGNED NOT NULL, item_name VARCHAR(50) NOT
NULL, amount INT DEFAULT 1, FOREIGN KEY (player_id) REFERENCES players(id) ON DELETE CASCADE, FOREIGN KEY
(item_name) REFERENCES items(name) ON DELETE CASCADE );

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

6. Insert data into items.

name durability item_type
Fire Bow 80 ranged
Iron Sword 100 melee
Magic Wand 60 magic
Shield 120 defense

Healing Potion 1 potion

Gun 90 ranged

Answer:

```
INSERT INTO items (name, durability, item_type) VALUES  
( 'Fire Bow', 80, 'ranged'),  
( 'Iron Sword', 100, 'melee'),  
( 'Magic Wand', 60, 'magic'),  
( 'Shield', 120, 'defense'),  
( 'Healing Potion', 1, 'potion'),  
( 'Gun', 90, 'ranged');
```

```
✓ 6 rows inserted. (Query took 0.0072 seconds.)  
  
INSERT INTO items (name, durability, item_type) VALUES ('Fire Bow', 80, 'ranged'), ('Iron Sword', 100, 'melee'),  
( 'Magic Wand', 60, 'magic'), ('Shield', 120, 'defense'), ('Healing Potion', 1, 'potion'), ('Gun', 90, 'ranged');  
  
[ Edit inline ] [ Edit ] [ Create PHP code ]
```

7. Insert data into players.

name health equipped_item Archer Alice 85 Fire Bow

Knight Karl 100 Iron Sword Mage Mira 70 Magic Wand Tank

Tom 120 Shield

Answer:

```
INSERT INTO players (name, health, equipped_item_name) VALUES  
( 'Archer Alice', 85, 'Fire Bow'),  
( 'Knight Karl', 100, 'Iron Sword'),  
( 'Mage Mira', 70, 'Magic Wand'),  
( 'Tank Tom', 120, 'Shield');
```

```
✓ 4 rows inserted.  
Inserted row id: 4 (Query took 0.0039 seconds.)  
  
INSERT INTO players (name, health, equipped_item_name) VALUES ('Archer Alice', 85, 'Fire Bow'), ('Knight Karl',  
100, 'Iron Sword'), ('Mage Mira', 70, 'Magic Wand'), ('Tank Tom', 120, 'Shield');  
  
[ Edit inline ] [ Edit ] [ Create PHP code ]
```

8. Insert data into inventory.

player_name item_name amount

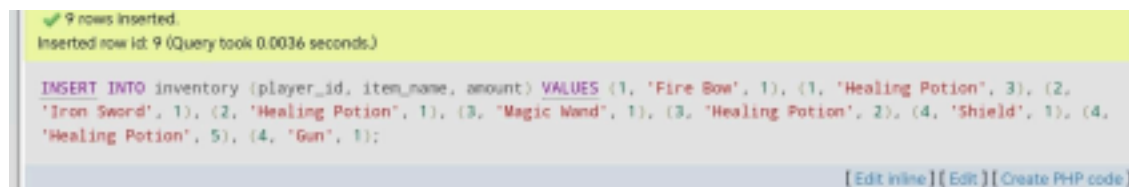
Archer Alice Fire Bow 1

Archer Alice Healing Potion 3

Knight Karl Iron Sword 1
Knight Karl Healing Potion 1
Mage Mira Magic Wand 1
Mage Mira Healing Potion 2
Tank Tom Shield 1
Tank Tom Healing Potion 5
Tank Tom Gun 1

Answer:

```
INSERT INTO inventory (player_id, item_id, amount) VALUES
(1, 1, 1),  -- Fire Bow
(1, 5, 3),  -- Healing Potion
(2, 2, 1),  -- Iron Sword
(2, 5, 1),  -- Healing Potion
(3, 3, 1),  -- Magic Wand
(3, 5, 2),  -- Healing Potion
(4, 4, 1),  -- Shield
(4, 5, 5),  -- Healing Potion
(4, 6, 1);  -- Gun
```

A screenshot of a database query execution interface. At the top, a green status bar indicates "9 rows inserted." and "Inserted row id: 9 (Query took 0.0036 seconds)". Below this, the SQL query is displayed in a monospaced font: `INSERT INTO inventory (player_id, item_name, amount) VALUES (1, 'Fire Bow', 1), (1, 'Healing Potion', 3), (2, 'Iron Sword', 1), (2, 'Healing Potion', 1), (3, 'Magic Wand', 1), (3, 'Healing Potion', 2), (4, 'Shield', 1), (4, 'Healing Potion', 5), (4, 'Gun', 1);`. At the bottom right of the code area, there are three links: "[Edit inline]", "[Edit]", and "[Create PHP code]".

```
✓ 9 rows inserted.
Inserted row id: 9 (Query took 0.0036 seconds)

INSERT INTO inventory (player_id, item_name, amount) VALUES (1, 'Fire Bow', 1), (1, 'Healing Potion', 3), (2,
'Iron Sword', 1), (2, 'Healing Potion', 1), (3, 'Magic Wand', 1), (3, 'Healing Potion', 2), (4, 'Shield', 1), (4,
'Healing Potion', 5), (4, 'Gun', 1);

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

Query

a) Display all players.

Answer:

```
SELECT * FROM players;
```

id	name	health	equipped_item_name
1	Archer Alice	85	Fire Bow
2	Knight Karl	100	Iron Sword
3	Mage Mira	70	Magic Wand
4	Tank Tom	120	Shield

b) Show all item names and their types.

Answer:

```
SELECT name, item_type FROM items;
```

name	item_type
Shield	defense
Magic Wand	magic
Iron Sword	melee
Healing Potion	potion
Fire Bow	ranged
Gun	ranged

c) Show all players with health greater than 90.

Answer:

```
SELECT name, health
```

```
FROM players
```

```
WHERE health > 90;
```

name	health
Knight Karl	100
Tank Tom	120

d) Show the inventory of each player.

Answer:

```
SELECT
    p.name AS player_name,
    i.name AS item_name,
    inv.amount
FROM
```

```

        inventory inv
JOIN
        players p ON inv.player_id = p.id
JOIN
        items i ON inv.item_id = i.id;

```

player_name	item_name	amount
Tank Tom	Shield	1
Mage Mira	Magic Wand	1
Knight Karl	Iron Sword	1
Archer Alice	Healing Potion	3
Knight Karl	Healing Potion	1
Mage Mira	Healing Potion	2
Tank Tom	Healing Potion	5
Archer Alice	Fire Bow	1
Tank Tom	Gun	1

e) Count the number of items in each player's inventory.

Answer:

```

SELECT
        p.name AS player_name,
        COUNT(*) AS item_count
FROM
        inventory inv
JOIN
        players p ON inv.player_id = p.id
GROUP BY
        p.id, p.name;

```

player_name	item_name	amount
Tank Tom	Shield	1
Mage Mira	Magic Wand	1
Knight Karl	Iron Sword	1
Archer Alice	Healing Potion	3
Knight Karl	Healing Potion	1
Mage Mira	Healing Potion	2
Tank Tom	Healing Potion	5
Archer Alice	Fire Bow	1
Tank Tom	Gun	1

f) Find players who have more than 2 potions.

Answer:

```

SELECT DISTINCT p.name
FROM inventory inv

```

```

JOIN players p ON inv.player_id = p.id
JOIN items i ON inv.item_id = i.id
JOIN item_types it ON i.item_type_id = it.id
WHERE it.name = 'potion' AND inv.amount > 2;

```

name
Archer Alice
Tank Tom

g) Show all items of type 'ranged'.

Answer:

```

SELECT * FROM items WHERE item_type = 'ranged';

```

name	durability	item_type
Fine Bow	80	ranged
Gun	90	ranged

h) Find the total number of items each player owns (sum of amounts).

Answer:

```

SELECT
    p.name AS player_name,
    SUM(inv.amount) AS total_items
FROM
    inventory inv
JOIN
    players p ON inv.player_id = p.id
GROUP BY
    p.id, p.name;

```

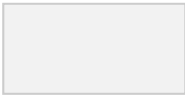


i) Find the most durable item.

Answer:

```
SELECT
    name,
    durability
FROM
    items
ORDER BY
    durability DESC;
```

```
LIMIT 1;
```



j) Find players who have more than one type of item.

Answer:

```
SELECT p.name
FROM inventory inv
JOIN players p ON inv.player_id = p.id
GROUP BY p.id, p.name
HAVING COUNT(DISTINCT inv.item_id) > 1;
```

